

```
$ devreport --render
```

devreport_

Turn a messy project folder into a polished PDF report or slide deck

```
// Celibistrial
```

```
// October 2026
```

\$ The problem

- > Student builders have to write up hackathons, coursework and side projects
- > They would rather keep building than write the report
- > Their material is scattered: dev logs, notes, CSVs, screenshots, a repo

\$ What it is

- > Drop in logs, notes, a CSV, screenshots, a `.zip` or a GitHub link
- > Plain code analyses the inputs into CSVs and `facts.json`
- > Four optional questions cover what the files can't answer
- > Claude Code writes and compiles a LaTeX report or Beamer deck
- > The PDF appears in the browser, with the `.tex` to download

\$ How it works: the pipeline

1. Upload, zip or GitHub link arrives at `server.js`
2. `collect.js` parses git history, logs, tables, notes, images
3. Optional answers are saved as `answers.md`
4. `claude -p` writes `main.tex` and compiles it with tectonic
5. `main.pdf` is shown inline; progress streams over SSE

\$ How it works: the pieces

- > `server.js`: uploads, intake, runs collector and Claude, streams progress
- > `collect.js`: deterministic parsing into `data/*.csv` and `facts.json`
- > `pptx.js`: reads a `.pptx` template's colours, fonts and layout
- > `index.html`: the whole UI in one file
- > The server uses only Node's standard library, no `npm install`

\$ Decision: charts can't lie

- > Every chart is pgfplots reading a CSV that `collect.js` computed
- > Claude never types a number into a chart
- > Stat numbers come from `facts.json`
- > Commit counts and lines of code are not treated as evidence

\$ Decision: our code analyses, Claude writes

- > Milestones come from git history, excluding vendored and generated code
- > The collector also picks likely entry points of the code
- > Recurring errors are grouped by normalizing numbers and IDs
- > Column types are detected for the user's own CSV or JSON
- > These steps are deterministic and covered by `node:test` tests

\$ Decision: a locked-down agent and safe intake

- > Claude runs headless with only Read, Write, Edit, tectonic and Skill
- > It can only write inside the job folder
- > Uploaded content is treated as data, never instructions
- > Zips are size-checked; path-escaping entries and symlinks are rejected
- > GitHub URLs are validated and cloned without a shell, with a timeout

\$ Themes and custom templates

- > Paper (serif, academic) and Midnight (dark, neon, terminal)
- > Each theme is one `.sty` for both reports and slides
- > Upload a `.pptx` and Claude writes a matching Beamer theme
- > It compiles a test slide and compares it with the template thumbnail

\$ Timeline (all on 2 October 2026)

- > Project plan and handoff notes
- > Server: multipart upload, zip and GitHub intake, Claude run, SSE progress
- > UI: drop zone, repo link, themes, questions, live feed, PDF view
- > devreport skill: pipeline rules, tested chart snippets, headless prompt
- > Paper and Midnight themes for reports and slides
- > Collector fixes: 16-bit PNGs, commas in CSV labels, day counts
- > .pptx templates for slides
- > Optional “proudest result” question as a source for Results

\$ Try it

- > `github.com/Celibistrial/devreport`
- > Needs macOS or Linux, Node 20+, Claude Code, tectonic, git, unzip
- > Run `node server.js` and open `127.0.0.1:3000`
- > Each run uses your own Claude Code login, about 1–3 minutes

Thank you